

Zero Day Initiative — Exploiting Exchange PowerShell After ProxyNotShell: Part 4

Zero Day Initiative — Exploiting Exchange PowerShell After ProxyNotShell: Part 4 - Piotr Bazydło, Zero Day Initiative.

- Published: 2024-09-26
- Original: <https://www.zerodayinitiative.com/blog/2024/9/25/exploiting-exchange-powershell-after-proxynotshell-part-4-no-argument-constructor>
- Preserved from: <https://www.zerodayinitiative.com/blog/2024/9/25/exploiting-exchange-powershell-after-proxynotshell-part-4-no-argument-constructor> (preserved-copy) on 2026-08-04
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source cited by ysonet, kept so the technique survives the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Blog

Exploiting Exchange PowerShell After ProxyNotShell: Part 4 - No Argument Constructor

** September 26, 2024

** Piotr Bazydło

As you may know, I recently presented my Exchange-related talk during OffensiveCon 2024. This series of 4 blog posts is meant to supplement the talk and provide additional technical details.

In this final part, I 'am going to describe the PowerShell Remoting

`ConvertViaNoArgumentConstructor` conversion mechanism, which I underestimated at the beginning of my research. It allowed me to find 3 more vulnerabilities, even after the Exchange PowerShell attack surface had been significantly hardened by switching to a strict allow list of types. The vulnerabilities I found were:

- CVE-2023-36050 - XXE (File Read) • CVE-2023-36039 - NTLM Relaying • CVE-2023-36035 - NTLM Relaying

At least at that time, Microsoft was taking seriously NTLM relaying vulnerabilities in Exchange, because it could lead to privilege escalation.

You can also watch the full talk here: [“Half Measures and Full Compromise: Exploiting Microsoft Exchange PowerShell Remoting”](#). This blog post covers the part from 26:25 to 29:45, but it significantly extends it.

Introduction

In this final part of the Exchange PowerShell series, I'll describe the `ConvertViaNoArgumentConstructor` conversion mechanism. Initially, I was not interested in it, as I was always able to find an applicable RCE chain using single-argument constructor conversion.

My approach changed when the new patch for my RCE chains landed. Finally, it was a good patch. The entire deserialization protection mechanism (type control) had been redesigned. The new mechanism:

- By default, works solely based on an allow list.
- The internal deserialization in `MultiValuedProperty` is subject to the same validation.
- Type parameters of generic classes (like class `T` in `SomeClass<T>`) are also subject to validation.

After this patch, we cannot longer take advantage of deserializing types that are missing from a deny list, since an allow list is being used instead. Also, the `MultiValuedProperty<T>` bridge gadget does not provide too much value, as the generic type parameter `<T>` is also subject to the same validation.

For my last dance with Exchange PowerShell, I decided to focus once more on the list of allowed classes. I had a hard time finding anything interesting for single-argument constructor or `Parse` method-based conversions, though. I decided to have a look at different conversions again, and I realized that the one based on a no-argument constructor is much more powerful than I initially thought.

The `ConvertViaNoArgumentConstructor`

`ConvertViaNoArgumentConstructor` works similarly to the majority of setter-based serializers:

- It initializes the object with a no-argument constructor.
- It uses setters to set the values of the members.

In the serialized XML payload, member values are defined using the `MS` tag:

[\[image: image\]](#)

*

Figure 1 - Sample member definition

*

The `MS` tag contains one subelement for each member that we want to deserialize. In the sample XML in Figure 1, we are specifying that:

- We want to deserialize a single member called `SomeMember`. The `N` attribute specifies the member name.
- We are deserializing the member value from a string. This is indicated by the `S` tag.
- The value for the member is the string `value`.

We can see that the `ConvertViaNoArgumentConstructor` conversion routine needs to somehow deserialize values for those setter calls. This is where things get interesting.

Let's start from the beginning. When we deserialize an object with `ConvertViaNoArgumentConstructor`, we eventually reach one of the `System.Management.Automation.LanguagePrimitives.`

`ConvertViaNoArgumentConstructor.Convert` overloads:

At [1], our object is instantiated with the no-argument constructor.

At [2], one of the `SetObjectProperties` overloads is called.

At [1], the code verifies that `psobject` is not `null`. This object holds data from our serialized payload, thus it should be a valid object.

At [2], it iterates over properties defined in our payload. It extracts the name of each member to be deserialized together with its value, and adds it to the dictionary at [3].

Finally, we reach another `SetObjectProperties` overload at [4].

At [1], it iterates over properties (members) specified in our serialized payload.

At [2], information about a given member is retrieved, including the type of the member in string form.

At [3], it calls `TypeResolver.TryResolveType` method to resolve the string representation of the type into a `Type` object.

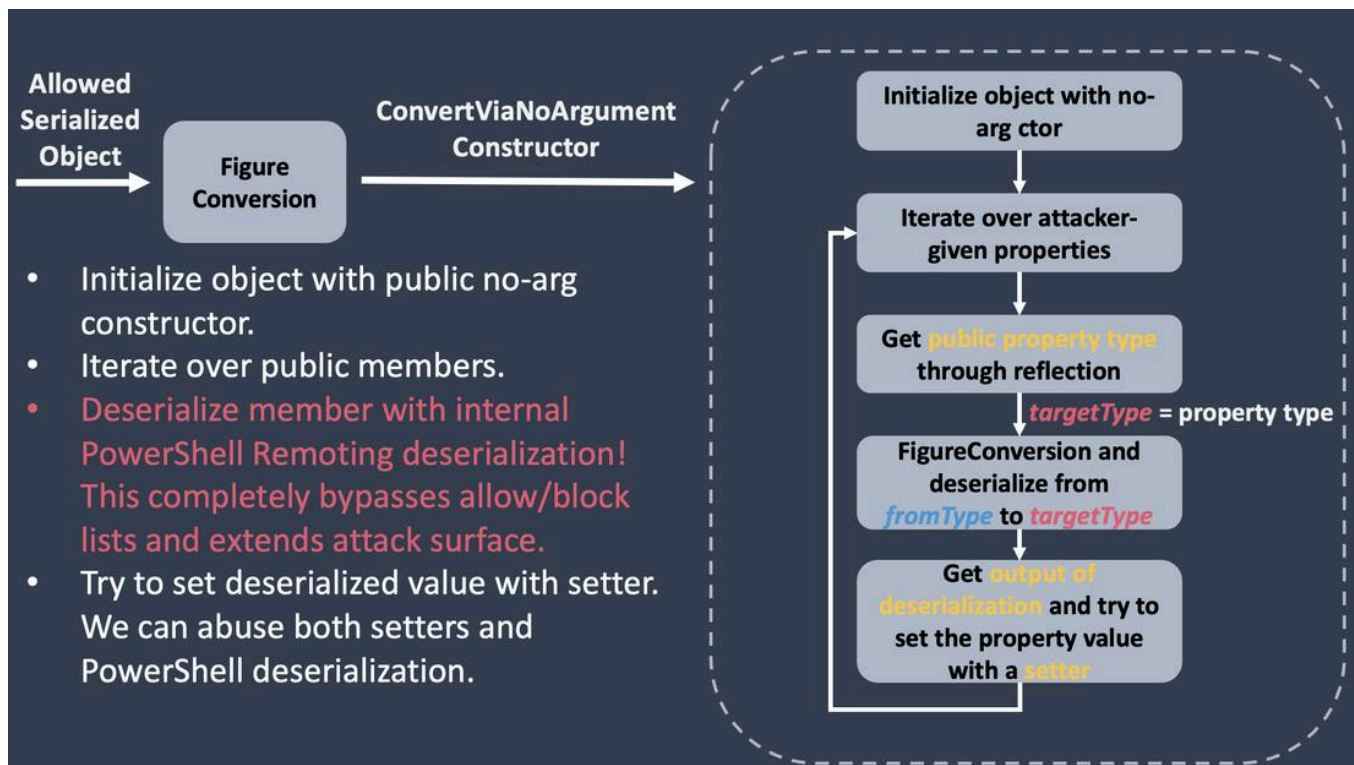
At [4], we reach the `LanguagePrimitives.ConvertTo` method, where:

- The target type is set to the type of the member that we want to deserialize.
- Our controlled value is provided as a value, from which we are going to deserialize!

To summarize: We can deserialize an allowed type through a no-argument constructor. We can set members of this type, using setter calls. When setting a member, PowerShell Remoting will use its internal conversions in `LanguagePrimitives.ConvertTo` to convert the value we specify. This means that it will try to deserialize it with all available conversions, including:

- Parse based conversion.
- Single-argument constructor conversion.
- No-argument constructor conversion.
- And others.

The entire process can be summarized with the following scheme.



*

Figure 2 - Simplified algorithm for ConvertViaNoArgumentConstructor

*

This deserialization routine opens up two main exploitation vectors:

a) Dangerous members of allowed types.

We can target types that:

- o are on the allow list (can be deserialized with PowerShell Remoting), and o have a no-argument constructor, and o have a public member of some dangerous type. By dangerous type, we mean that it leads to security impact when deserialized with PowerShell Remoting. It is not necessary for this type to be on the allow list.

It is also important to note that deserialization of a member will happen even if the member has no setter, or even if there is a setter but the setter is not public!

b) Dangerous setters of allowed types

We can also target the setters themselves if the implemented setter leads to something dangerous. I'll show one example of this later.

ConvertViaNoArgumentConstructor - Example

You can see that this conversion mechanism is extremely hazardous because it allows us to significantly extend the attack surface. This is because members of allowed types can be deserialized with PowerShell Remoting conversions, even when the member of is not of an allowed type. Let's see how it works based on some simple examples.

Consider the following sample class, and suppose that it is on the list of types allowed to be deserialized with PowerShell Remoting.

```
public class someWhitelistedType
{
    public someWhitelistedType()
    {
    }

    public string Name { get; set; }

    public Int32 count { get; set; }

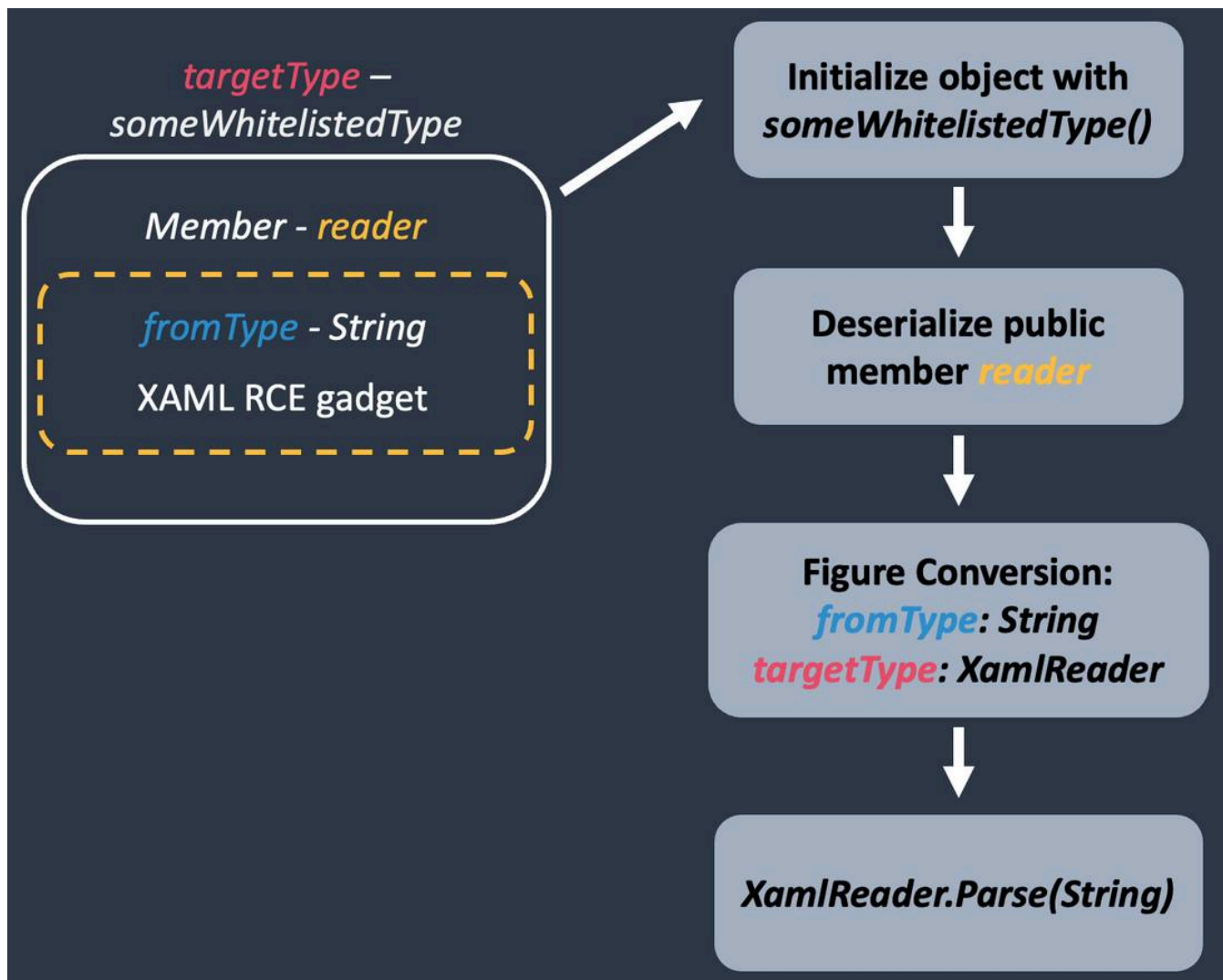
    public XamlReader reader { get; set; }
}
```

*

Figure 3 - Sample allowed class

*

We can see that it contains a no-argument constructor. Moreover, it contains a member `reader`, which is of type `XamlReader`. We can deliver a serialized object of type `someWhitelistedType`, and we can deserialize the `reader` member from a string. The deserialization flow is presented in the next figure.



*

Figure 4 - Deserialization of sample `someWhitelistedType` class

*

As expected, this leads to RCE. This is because PowerShell Remoting will try to deserialize a member of type `XamlReader`. `XamlReader` has already been abused in ProxyNotShell and other exploit chains to achieve RCE through XAML deserialization.

You could also imagine building a chain of types and members, where we are nesting multiple no-argument conversions to ultimately reach a member with a dangerous type.



*
 Figure 5 - Chaining multiple no-argument constructor conversions to reach dangerous member
 *

Now that we know how this powerful conversion works, let's have a look at how I was able to leverage it to abuse three allowed Exchange classes.

CVE-2023-36039 - NTLM Relaying with FederationTrust

The first class I abused was

`Microsoft.Exchange.Data.Directory.SystemConfiguration.FederationTrust`, which was allowed in Exchange PowerShell Remoting. This class has an interesting public member, called `OrgCertificate`.

```

public X509Certificate2 OrgCertificate
{
    get
    {
        return this[FederationTrustSchema.OrgCertificate] as X509Certificate2;
    }
    internal set
    {
        this[FederationTrustSchema.OrgCertificate] = value;
    }
}
  
```

*
 Figure 6 - OrgCertificate member of FederationTrust
 *

It is of type `X509Certificate2`. This type is not allowed in Exchange PowerShell, but we can still deserialize it when it is the type of a member of an allowed class.

It turns out that this type implements a single-argument constructor, which accepts a path to the certificate file and tries to read it.

[image: image]

*

Figure 7 - X509Certificate2 constructor

*

We can reach this constructor through PowerShell Remoting single-argument constructor conversion and use it for NTLM relaying.

CVE-2023-36050 - XXE with TransportConfigContainer

Another class is

`Microsoft.Exchange.Data.Directory.SystemConfiguration.TransportConfigContainer`. Here, I abused the second kind of exploitation vector: attacking the setter call itself.

This class defines a `TransportSystemState` member, which is of type `string`.

```
public string TransportSystemState
{
    get
    {
        return (string)this[TransportConfigContainerSchema.TransportSystemState];
    }
    set
    {
        this[TransportConfigContainerSchema.TransportSystemState] = value;
        this.RefreshTransportSystemState();
    }
}
```

*

Figure 8 - TransportSystemState member

*

We obviously can't achieve anything malicious during the deserialization of `string`. However, when setting the property, the setter calls `RefreshTransportSystemState`, and this leads to XXE. This is because our input to the setter eventually reaches the `GetOverrides` method.

```
public static IEnumerable<TransportSystemStateOverride> GetOverrides(string rawTransportSystemState)
{
    if (string.IsNullOrEmpty(rawTransportSystemState))
    {
        return null;
    }
    XmlDocument xmlDocument = new XmlDocument();
    xmlDocument.LoadXml(rawTransportSystemState);
    ...
}
```

*

Figure 9 - GetOverrides - XXE

*

Exchange runs on an older version of .NET Framework, where `XmlDocument` is not protected by default from XXE vulnerabilities (classic). As we control the XML string through deserialization, we can abuse this, for example to read local files.

CVE-2023-36035 - NTLM Relaying (Partial Bypass for CVE-2023-36756 RCE)

The last vulnerability is a partial bypass for CVE-2023-36756. This vulnerability was described in the [second part](#) of this series. A small recap:

- We were able to deliver a CAB file to the Exchange using a UNC path (a path to a public SMB share within the domain).
- Exchange used the Windows `extrac32` utility for CAB extraction, which is vulnerable to path traversal.
- We could upload a webshell and get code execution.

This vulnerability was patched by making three changes:

- `ApprovedApplicationCollection` is no longer allowed. This is the class that was used to reach the CAB extraction.
- `extrac32` is no longer used for the extraction.
- A call to `IsUNCPath` was added, which should block the extraction of CAB files from an attacker's share.

Let's have a look at this `IsUNCPath` method:

View fullsize

[\[image: image\]](#)

*

Figure 10 - IsUNCPath method

*

It first uses `Uri.TryCreate` to try parsing the file path, and if that succeeds, it verifies if we have delivered a UNC path by calling `Uri.IsUnc`. If at least one of those conditions is not true, we would be able to bypass this check. Although the path traversal vulnerability is no longer present, meaning that this would no longer allow us to upload a web shell, nevertheless it at least could be used for NTLM relay.

It turns out that the bypass is very simple here. We can just use the `\\?\UNC\server\...` syntax (if you don't know it, please see [this article](#) by James Forshaw).

```
Uri.TryCreate("\\\\?\UNC\win-attacker\poc") -> null
```

The `Uri` class is not able to handle this syntax and it will return `null` ! However, this path will be later used in the `FileInfo` constructor:

```
new FileInfo("\\\\?\UNC\win-attacker\poc") -> OK
```

Using this syntax, we can bypass the `IsUNCPath` check and potentially perform NTLM relaying.

We are still missing one piece, though. How can we reach the CAB extraction methods, if our `ApprovedApplicationCollection` type was removed from the list of allowed classes?

`ConvertViaNoArgumentConstructor` to the rescue! It turned out that Exchange allows deserializing the `MobileMailboxPolicy` class. This class has a member which is of type `ApprovedApplicationCollection` !

```
[Serializable]
public class MobileMailboxPolicy : MailboxPolicy
{
    ...
    public ApprovedApplicationCollection ApprovedApplicationList
    {
        get
        {
            return (ApprovedApplicationCollection)this[MobileMailboxPolicySchema.ADApprovedApplicationList];
        }
        set
        {
            this[MobileMailboxPolicySchema.ADApprovedApplicationList] = value;
        }
    }
    ...
}
```

*

Figure 11 - MobileMailboxPolicy member of ApprovedApplicationCollection type

*

Even though Microsoft didn't want us to deserialize `ApprovedApplicationCollection` anymore, we were still able to do just that through member deserialization. This shows the power of the no-argument constructor conversion.

Summary

In this blog post, I presented the `ConvertViaNoArgumentConstructor` conversion of PowerShell Remoting. It is a powerful deserialization mechanism, which significantly extends the attack surface for the entire PowerShell Remoting. It helped me to abuse 3 more allowed Exchange classes, to gain quite a good impact: NTLM Relaying (Privilege Escalation) and XXE (File Read + NTLM Relaying).

This will be the last post in the Exchange PowerShell Remoting series, and I hope you enjoyed it. In my upcoming blog posts, I'm planning to focus on some vulnerabilities that I have recently discovered in Microsoft SharePoint, so stay tuned.

Until my next post, you can follow me [@chudypb](#) and follow the team on [Twitter](#), [Mastodon](#), [LinkedIn](#), or [Bluesky](#) for the latest in exploit techniques and security patches.

- [Microsoft](#)
- [Exchange](#)
- [Research](#)



Stand at the front line of proactive security

TrendAI™ ZDI connects the experts who discover, remediate, and defend. Add your voice to the work that pushes attackers back.

[

RESEARCHERS

Archived from <https://zerodayinitiative.com/blog/2024/9/25/exploiting-exchange-powershell-after-proxynotshell-part-4-no-argument-constructor>. Rendered offline from the local Markdown copy.